

集训项目2：图片识别与动作通信

Task1：图片识别

图片识别任务目录：

```
/home/unitree/training_camp/module_8_d1/task_1_image_recognition
```

待识别图片：

```
/home/unitree/training_camp/module_8_d1/task_1_image_recognition/images/action.png
```

待完善程序文件：

```
image_check.sh  
image_check.py  
image_check.cpp
```

任务要求

1. 打开终端，进入图片识别任务目录。
2. 使用 Visual Studio Code 或其他编辑工具打开指定程序文件。
3. 阅读程序逻辑，理解程序需要读取的图片路径和输出内容。
4. 补全程序中的空缺部分，使程序能够正确读取指定图片。
5. 运行程序，确认终端能够输出图片路径、文件大小、图片类型和提示信息。
6. 对完整代码和终端运行结果进行截图，并粘贴到答题报告对应区域。

运行说明

进入任务目录：

```
cd ~/training_camp/module_8_d1/task_1_image_recognition
```

运行 Shell 版本：

```
bash image_check.sh
```

运行 Python 版本：

```
python3 image_check.py
```

编译并运行 C++ 版本：

```
g++ image_check.cpp -o image_check  
./image_check
```



完成目标

程序能够正常运行，终端能够打印指定图片的识别结果，运行过程中不出现文件不存在、读取失败或函数调用错误。

Task2：动作通信

动作通信任务目录：

```
/home/unitree/training_camp/module_8_d1/task_2_placeholder
```

ROS 工作空间目录：

```
/home/unitree/training_camp/module_8_d1/task_2_placeholder/work_2_2_ws/work_2_2_ws
```

功能包目录：

```
/home/unitree/training_camp/module_8_d1/task_2_placeholder/work_2_2_ws/work_2_2_ws/src/demo_action
```

Action 定义文件：

```
src/demo_action/action/AddInts.action
```

待完善程序文件：

```
src/demo_action/scripts/action_server.py
src/demo_action/scripts/action_client.py
```

工程配置文件：

```
src/demo_action/CMakeLists.txt
src/demo_action/package.xml
```

阅读 Action 定义文件，理解目标值、最终结果和过程反馈的通信结构，再补全服务端和客户端程序。

本任务中需要补全一个基于 ROS 1 actionlib 的动作通信示例，使服务端能够接收客户端目标、执行计算过程、发布过程反馈，并在完成后返回最终结果。

任务要求

1. 进入动作通信 ROS 工作空间。
2. 查看 AddInts.action 文件，理解目标、结果和反馈三部分内容。
3. 打开 action_server.py，补全服务端核心逻辑。
4. 打开 action_client.py，补全客户端核心逻辑。
5. 编译工作空间并刷新环境变量。
6. 启动 roscore。
7. 启动 Action 服务端，确认服务端处于等待目标状态。



8. 启动 Action 客户端，发送目标并观察通信过程。
9. 确认终端能够显示服务激活、过程反馈和最终结果。
10. 对完整代码和终端运行结果进行截图，并粘贴到答题报告对应区域。

操作说明

进入 ROS 工作空间：

```
cd ~/training_camp/module_8_d1/task_2_placeholder/work_2_2_ws/work_2_2_ws
```

编译工作空间：

```
catkin_make
```

刷新环境变量：

```
source devel/setup.bash
```

如脚本无法执行，可检查脚本权限：

```
chmod +x src/demo_action/scripts/action_server.py
chmod +x src/demo_action/scripts/action_client.py
```

运行说明

终端 1 启动 ROS master：

```
roscore
```

终端 2 启动 Action 服务端：

```
cd ~/training_camp/module_8_d1/task_2_placeholder/work_2_2_ws/work_2_2_ws
source devel/setup.bash
roslaunch demo_action action_server.py
```

终端 3 启动 Action 客户端：

```
cd ~/training_camp/module_8_d1/task_2_placeholder/work_2_2_ws/work_2_2_ws
source devel/setup.bash
roslaunch demo_action action_client.py
```

完成目标

服务端和客户端能够正常建立 Action 通信。客户端发送目标后，服务端能够执行任务并发布过程反馈；任务完成后，客户端能够接收到最终结果。运行过程中不应出现模块导入失败、Action 类型找不到、服务端连接失败或程序异常退出等问题。



Python 版本修改说明

本文说明如何把 `image_check.py` 从任务中的错误状态修改为可运行答案。

目标

程序需要读取图片文件：

```
/home/unitree/training_camp/module_8_d1/task_1_image_recognition/images/action.png
```

然后判断图片格式，并输出识别结果和信息。

第 1 处：补全图片路径

任务中给出的路径只有目录：

```
DEFAULT_IMAGE = Path("/home/unitree/training_camp/module_8_d1/")
```

这里只指向了 `module_8_d1` 文件夹，不是具体图片文件，所以程序会判断文件不存在。

需要继续找到图片所在位置：

```
task_1_image_recognition/images/action.png
```

修改为：

```
DEFAULT_IMAGE =  
Path("/home/unitree/training_camp/module_8_d1/task_1_image_recognition/images/action.png")
```

第 2 处：使用二进制方式打开图片

错误位置：

```
with image_path.open("____") as image_file:
```

图片是二进制文件，不能按普通文本方式读取，这里应该使用 `rb`，表示 read binary。

修改为：

```
with image_path.open("rb") as image_file:
```

第 3 处：调用图片类型识别函数

错误位置：

```
image_type = ____ (header)
```

前面已经定义了识别函数：



```
def detect_image_type(header: bytes) -> str:
```

所以这里应该调用 `detect_image_type` , 并把读取到的文件头 `header` 传进去。

修改为:

```
image_type = detect_image_type(header)
```

运行方式

在 Ubuntu 中运行:

```
python3 image_check.py
```

如果修改正确, 会输出图片路径、文件大小、图片类型和祝福信息。



C++ 版本修改说明

本文说明如何把 `image_check.cpp` 从任务中的错误状态修改为可运行答案。

目标

程序需要读取图片文件：

```
/home/unitree/training_camp/module_B_d1/task_1_image_recognition/images/action.png
```

然后读取文件头，判断图片格式，并输出结果。

第 1 处：补全图片路径

任务中给出的路径只有目录：

```
const std::string DEFAULT_IMAGE = "/home/unitree/training_camp/module_B_d1/";
```

这里只是文件夹路径，不是图片文件路径。程序最终要打开的是 `action.png`，所以要补全后面的目录和文件名。

修改为：

```
const std::string DEFAULT_IMAGE =  
"/home/unitree/training_camp/module_B_d1/task_1_image_recognition/images/action.png";
```

第 2 处：使用二进制方式打开图片

错误位置：

```
std::ifstream imageFile(imagePath, ____);
```

C++ 读取图片时应该使用二进制模式，否则文件内容可能被当作普通文本处理。

这里应该填写：

```
std::ios::binary
```

修改为：

```
std::ifstream imageFile(imagePath, std::ios::binary);
```

第 3 处：调用图片类型识别函数

错误位置：

```
std::string imageType = ____ (header);
```



前面已经定义了函数：

```
std::string detectImageType(const std::vector<unsigned char>& header)
```

所以这里应该调用 `detectImageType`，把读取出来的 `header` 传入。

修改为：

```
std::string imageType = detectImageType(header);
```

编译和运行

在 Ubuntu 中运行：

```
g++ image_check.cpp -o image_check  
./image_check
```

如果修改正确，会输出图片路径、文件大小、图片类型和信息。



Shell 版本修改说明

本文说明如何把 `image_check.sh` 从任务中的错误状态修改为可运行答案。

目标

脚本需要读取图片文件：

```
/home/unitree/training_camp/module_8_d1/task_1_image_recognition/images/action.png
```

然后读取图片文件头，判断图片格式，并输出结果。

第 1 处：补全图片路径

这里只指向了 `module_8_d1` 目录，不是具体的图片文件。需要继续找到图片所在的任务目录、图片目录和文件名。

修改为：

```
DEFAULT_IMAGE="/home/unitree/training_camp/module_8_d1/task_1_image_recognition/images/action.png"
```

I

第 2 处：判断路径是否是普通文件

错误位置：

```
if [ ! ____ "$IMAGE_PATH" ]; then
```

Shell 中 `-f` 用来判断路径是否存在，并且是否是普通文件。

修改为：

```
if [ ! -f "$IMAGE_PATH" ]; then
```

第 3 处：使用命令替换保存文件头

错误位置：

```
HEADER_HEX=____(od -An -tx1 -N16 "$IMAGE_PATH" | tr -d ' \n')
```

Shell 中需要用 `$( )` 执行命令，并把命令输出保存到变量中。

这里缺少的是 `$`，

修改为：

```
HEADER_HEX=$(od -An -tx1 -N16 "$IMAGE_PATH" | tr -d ' \n')
```




```
HEADER_HEX=$(od -An -tx1 -N16 "$IMAGE_PATH" | tr -d ' \n')
```

运行方式

在 Ubuntu 中运行:

```
bash shell 文件
```

如果修改正确, 会输出图片路径、文件大小、图片类型和信息。



ROS Action 通信代码补全指南

本文档旨在指导如何补全基于 ROS1 `actionlib` 的 Action 通信示例代码。内容涵盖服务端 (Server) 与客户端 (Client) 的核心逻辑补全，并附带完整代码对照及运行提示。

一、服务端 (Server) 代码补全

服务端的核心任务是接收目标、执行耗时任务、发布过程反馈，并在任务完成后返回最终结果。

1. 补全回调函数 `cb` 的初始化

在执行循环之前，需要从传入的 `goal` 中提取目标数值，并初始化累加变量和频率控制对象。

```
def cb(self, goal):  
    # 1. 获取目标值并初始化变量  
    num = goal.num  
    sum_val = 0  
    rate = rospy.Rate(1) # 假设以 1Hz 的频率进行累加和反馈  
    rospy.loginfo("接收到目标数值: %d", num)
```

2. 补全返回最终结果

在循环结束后，需要创建一个结果对象，将计算好的 `sum_val` 赋值给它，并调用 `set_succeeded` 告诉客户端任务已成功完成。

```
# 2. 返回最终结果  
result = AddIntsResult()  
result.result = sum_val  
self.server.set_succeeded(result)  
rospy.loginfo("计算完成, 最终结果: %d", sum_val)
```

二、客户端 (Client) 代码补全

客户端的核心任务是创建 Action 客户端对象、组装目标消息、发送目标并绑定相关的回调函数。

1. 创建 Action 客户端

在 `if __name__ == "__main__":` 块中，使用 `actionlib.SimpleActionClient` 创建客户端。需要指定服务端的 Action 名称（需与服务端一致）以及消息类型。

```
# 1. 创建 Action 客户端  
client = actionlib.SimpleActionClient("add_ints_action", AddIntsAction)
```



```
client.wait_for_server() # 阻塞等待服务端启动
rospy.loginfo("服务端已连接，准备发送目标...")
```

2. 组织目标并发送

实例化目标消息对象，填入需要计算的数据，然后调用 `send_goal` 方法，将前面定义好的三个回调函数（`done_cb`, `active_cb`, `fb_cb`）作为参数传入。

```
# 2. 组织目标并发送
goal = AddIntsGoal()
goal.num = 100 # 设置目标数，例如计算 1 到 100 的累加

# 发送目标，并绑定三个回调函数
client.send_goal(goal, done_cb=done_cb, active_cb=active_cb,
feedback_cb=fb_cb)
```

运行提示

- 先打开一个终端，输入 `roscore`
- 确保你的工作空间已经编译（`catkin_make`）。
- 在运行前 `source` 了环境，先在终端 1 运行服务端：`roslaunch demo_action action_server.py`。
- 在运行前 `source` 了环境，再在终端 2 运行客户端：`roslaunch demo_action action_client.py`。
- 你将看到客户端持续打印进度反馈，并在最后打印最终计算结果。

